

CPRE 431

Module 2 HW

Assignments will be submitted in PDF format via Canvas.

Please submit your homework online through Canvas. Late homework will not be accepted.

Important: Your submission must be in .pdf format ONLY!

You are not allowed to use AI tools to answer any of the HW questions

- 1. Suppose that your friend suggests the following way to confirm that both of you have the same secret key. You create a random bit string which is having the length of the key, then, XOR it with the key, and send the result over the channel. Your partner XORs the incoming block with the key (which should be the same key) and sends the result back. If what you receive is your original string, you have verified that your partner has the same secret key, yet neither of you has ever transmitted the key on the channel. Is there a flaw in this scheme?**

This way seems to be secure at first glance when I worked it out on my own. Since the key is never really sent from one person to the other, it seems like a good idea. If we say the random bit string is S and let the key be K . Then the first person XORs S and K to get a result (R). Then when R is sent to his friend, he XORs R with K to get a result S . He then sends S back to the first person who is able to confirm that this was his original string.

The main flaw in using this scheme is that someone who may be sitting in the middle (Man in the middle) will see both the messages sent over. First he will see the R that the first person sent, then he will see the S that the second person sent back. Having both the R and S in his hands, the attacker (if he is smart enough, and he knows what you are doing) simply has to XOR R and S together to retrieve the K that you and your partner have been using. XOR is an associative and commutative function which is the reason why this method is actually weak.

2. Fill in the table below, according to the following assumptions:

Modern computers run approximately 3 billion compute cycles per second and with cloud computing, it is easily imaginable that a well-funded private organization can rent 100,000 such systems in the cloud. To test whether a given ciphertext decrypted with a key matches a known plaintext, takes approximately 1000 compute cycles.

- a. Assuming that we have a block of ciphertext with known plaintext, indicate the time in years needed to test every possible key for each key length, in the case of:**
- i. One modern computer.**

Per Modern Computer: $3 * 10^9$, and with testing: $\frac{3 * 10^9}{1000} = 3 * 10^6 \text{ keys/second}$

Then for converting from seconds to years, we divide by $(60 * 60 * 24 * 365)$ in the calculations.

Key Length (bits)	Number of Keys	Attack Time (Years) (Worst case scenario)
56	2^{56}	$\frac{2^{56}}{3 * 10^6} = 278 \text{ days} = 761 \text{ years}$
128	2^{128}	$\frac{2^{128}}{3 * 10^6} = 3.60 * 10^{24} \text{ years}$
256	2^{256}	$\frac{2^{256}}{3 * 10^6} = 1.22 * 10^{63} \text{ years}$
512	2^{512}	$\frac{2^{512}}{3 * 10^6} = 1.42 * 10^{140} \text{ years}$

- ii. A well-funded private organization as described above.**

Per Modern Computer: $3 * 10^9$, With cloud computing: $3 * 10^9 * 100,000$

Per Modern Computer, with cloud computing, and testing: $\frac{3 * 10^9 * 100,000}{1000} = 3 * 10^{11} \text{ keys/second}$

Then for converting from seconds to years, we divide by $(60 * 60 * 24 * 365)$ in the calculations.

Key Length (bits)	Number of Keys	Attack Time (Years) (Worst case scenario)
56	2^{56}	$\frac{2^{56}}{3 * 10^{11}} = 2.7 \text{ days} = 0.00739 \text{ years}$
128	2^{128}	$\frac{2^{128}}{3 * 10^{11}} = 3.65 * 10^{19} \text{ years}$
256	2^{256}	$\frac{2^{256}}{3 * 10^{11}} = 1.22 * 10^{58} \text{ years}$
512	2^{512}	$\frac{2^{512}}{3 * 10^{11}} = 1.42 * 10^{135} \text{ years}$

- b. Assume you were working on a research lab, and you were asked to find a solution to improve the cracking performance and time. Your team came up with two promising solutions.
- i. Use specialized hardware to increase the speed of such an attack by 1000 times.
 - ii. Make cryptanalysis of the cipher to be able to guess half of the key bits at little cost.

Explain with calculations which solution do you think would give better results?

While solution (i) seems to be a good idea, increasing the speed of such an attack by 1000 times makes nearly no difference in the grand scheme of things. For example, while it may improve the Attack time for 56 bits from 761 years to 278 days, assuming the key/sec speed is now $3 * 10^9$ instead of $3 * 10^6$ from our part 2(a)(i), this seems as though it goes down from 761 years to 278 days so it is a good solution. But the moment our number of bits go up to 128, it still takes $3.59 * 10^{21}$ years with our speed being 1000 times faster.

In contrast, using cryptanalysis and being able to guess half of the key bits is a significantly better solution out of the two.

This solution would give better results because if we are able to somehow guess half of the key bits, then using example 2(a)(i) with 56 bits (halved to 28), with our attack speed still at $3 * 10^6$ we reduce our time from 761 years to 1.49 minutes. Even using the same example but with 128 bits (halved to 64), attack speed still at $3 * 10^6$ we reduce our time from $3.60 * 10^{24}$ years to 194,980 years. Yes, 194,980 years is still a long time, but in the grand scheme of things we were able to bring down the time from $3.60 * 10^{24}$ to $195 * 10^3$ years.

All in all, it is clear that the better results are given by solution (ii).